Solution: Remove All Adjacent Duplicates in String II

Problem Statement

You are given a string `s` and an integer `k`. Your task is to remove groups of identical, consecutive characters from the string such that each group has exactly `k` characters. The removal of groups should continue until it's no longer possible to make any more removals. The result should be the final version of the string after all possible removals have been made.

Examples

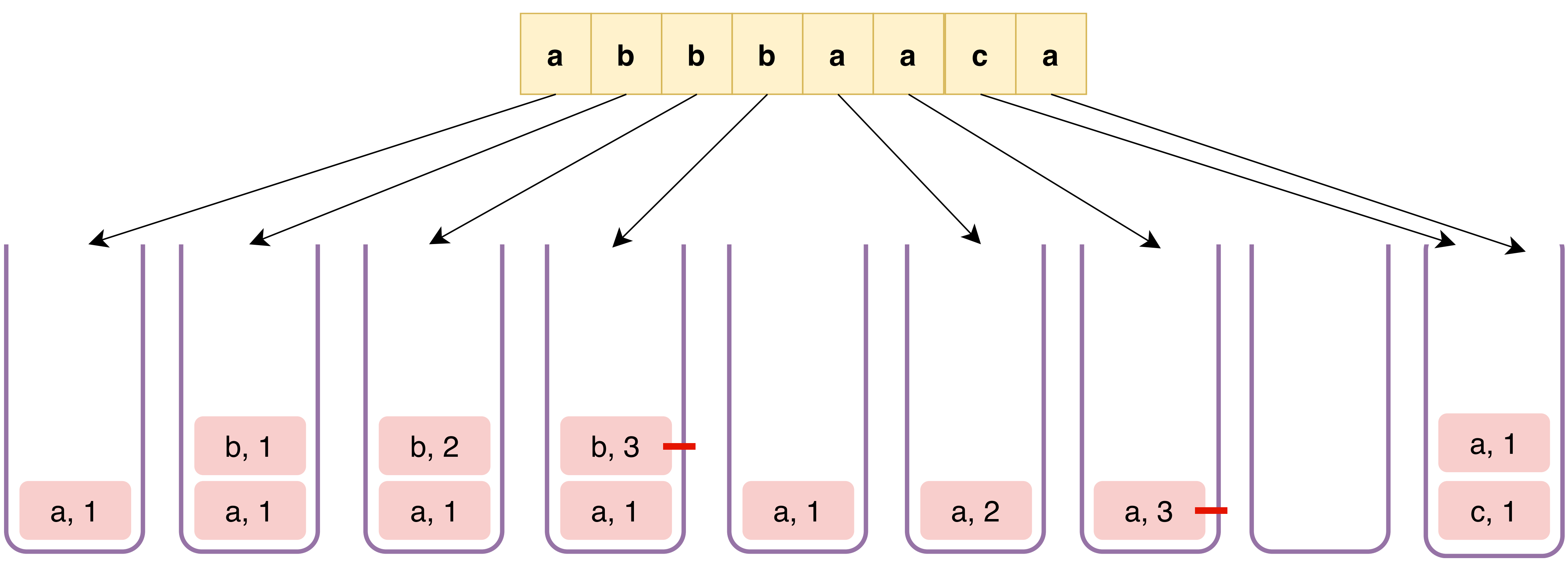
1. - Input: `s = "abbbaaca", k = 3`
   - Output: `"ca"`
   - Explanation: First, we remove "bbb" to get "aaaca". Then, we remove "aaa" to get "ca".
2. - Input: `s = "abbaccaa", k = 3`
   - Output: `"abbaccaa"`
   - Explanation: There are no instances of 3 adjacent characters being the same.
3. - Input: `s = "abbacccaa", k = 3`
   - Output: `"abb"`
   - Explanation: First, we remove "ccc" to get "abbaaa". Then, we remove "aaa" to get "abb".

Solution

This problem can be solved by using a stack to keep track of the characters and their counts. We iterate through the string and add each character and its count to the stack. If the count of the top character of the stack becomes `k`, we remove that entry from the stack. The result string is then constructed from the remaining characters in the stack.

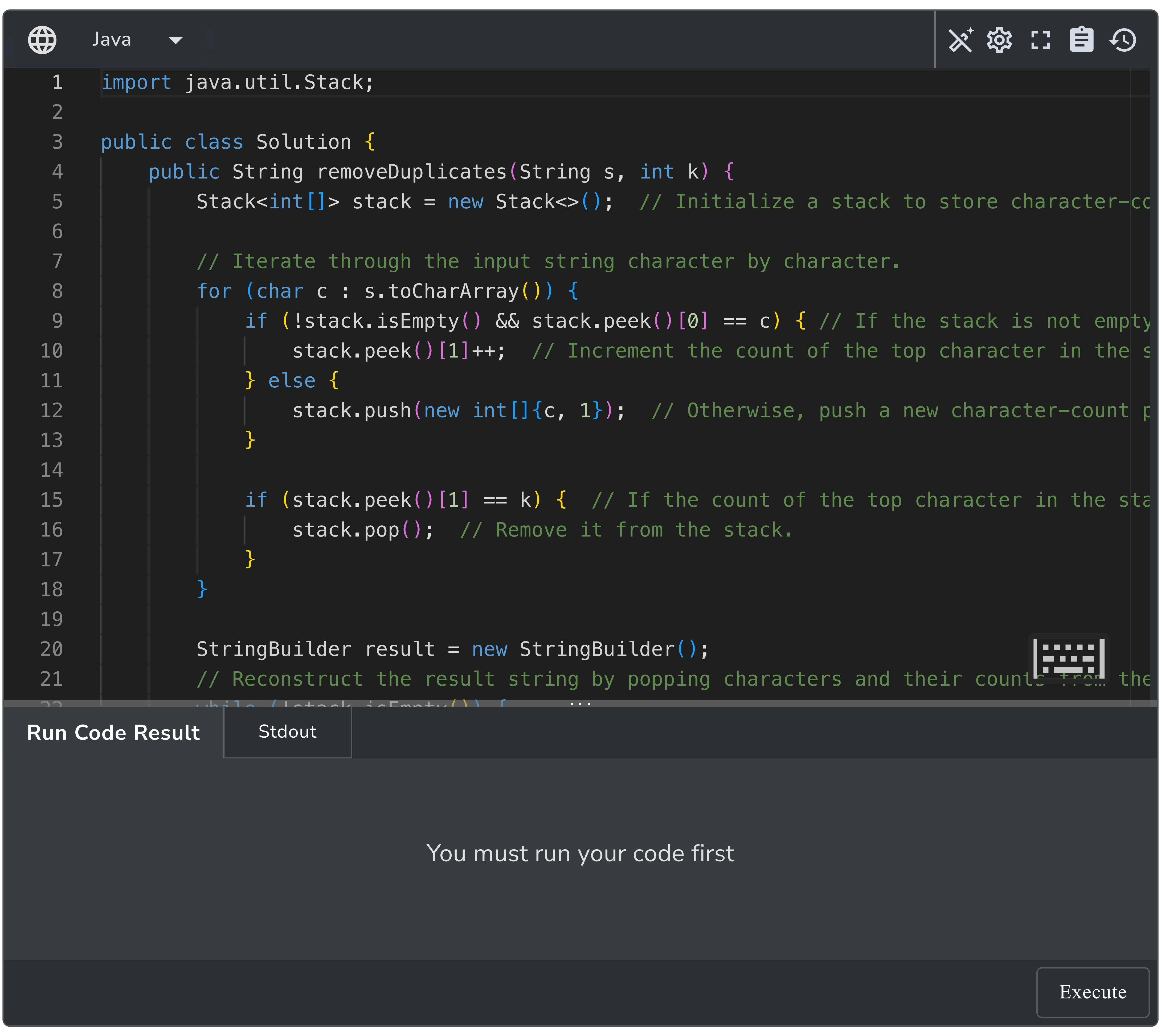
Algorithm Walkthrough

1. Initialize an empty stack. Each entry in the stack is a pair consisting of a character and its count.
2. Loop through the characters in `s`.
3. For each character:
   - If the stack is not empty and the current character is the same as the top character on the stack, increment the count of the top character.
   - Otherwise, add the current character and a count of 1 to the stack.
   - If the count of the top character on the stack is `k`, pop the top character from the stack.
4. Build the result string from the characters remaining on the stack.



Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;

public class Solution {
    public String removeDuplicates(String s, int k) {
        Stack<int[]> stack = new Stack<>();  // Initialize a stack to store character-cc

        // Iterate through the input string character by character.
        for (char c : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek()[0] == c) { // If the stack is not empty
                stack.peek()[1]++;  // Increment the count of the top character in the s
            } else {
                stack.push(new int[]{c, 1});  // Otherwise, push a new character-count p
            }

            if (stack.peek()[1] == k) {  // If the count of the top character in the sta
                stack.pop();  // Remove it from the stack.
            }
        }

        StringBuilder result = new StringBuilder();
        // Reconstruct the result string by popping characters and their counts from the
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is $O(N)$, where N is the length of `s`, as we perform one operation per character. The space complexity is also $O(N)$, as in the worst case, all characters are pushed onto the stack.

← Previous
Remove All Adjacent Duplicates In String II (Medium)

Next →
Remove K Digits (Hard)

☐ Mark as Completed